

# FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of Mon 13 JUNE 2022 – Theory Part

**WITH SOLUTIONS** – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

---

(capital letters please)

MATRICOLA:

---

(or PERSON CODE)

SIGNATURE:

---

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): syntax and semantics of languages, divided in four sections:
    - regular expressions and finite automata
    - free grammars and pushdown automata
    - syntax analysis and parsing methodologies
    - language translation and semantic analysis
  2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be  $\geq 18/30$ .
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

## 1 Regular Expressions and Finite Automata 20%

1. Consider the following regular expression  $R$  over a three-letter alphabet  $\{ a, b, c \}$ :

$$R = ( a \mid c a ) ( b a )^*$$

Answer the following questions:

- (a) List all the strings of length 4 and 5 that are generated by the given regular expression  $R$ .
  - (b) By using the Berry-Sethi method (BS), from the regular expression  $R$  obtain an equivalent finite-state automaton  $A$ . Minimize this automaton  $A$ , if necessary.
  - (c) List the computations of the automaton  $A$  of point (b) for the strings of point (a) above.
  - (d) From the automaton  $A$  of point (b), derive an equivalent right-linear grammar.
  - (e) By using the node elimination method (BMC), re-obtain the original regular expression  $R$  (or an equivalent one) from the automaton  $A$  of point (b). Start from the minimized version of automaton  $A$ , if it was necessary to minimize it.
  - (f) (optional) Say if the language  $L(R)$  is local and suitably justify your answer.
-

## Solution

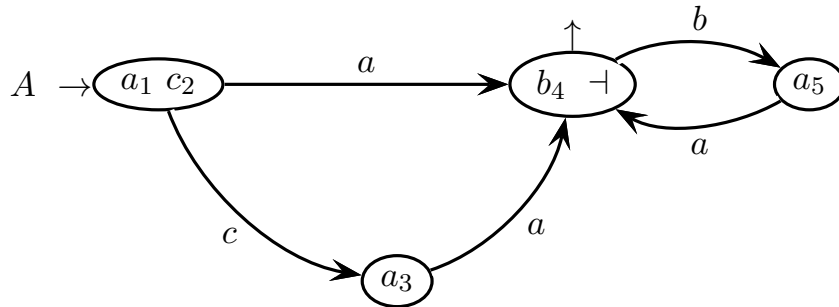
- (a) Here are the strings of length 4 and 5 generated by the regular expression  $R$ :

$c a b a \quad a b a b a$

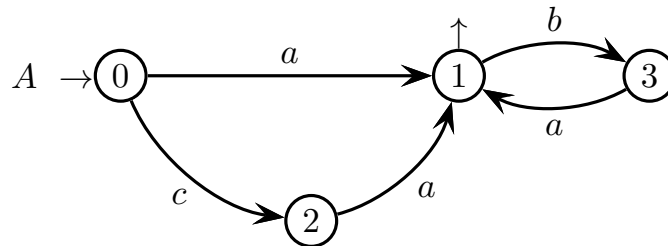
In general the valid strings start either with a single letter  $a$  or with a letter pair  $ca$ , and optionally continue with one or more letter pairs  $ba$ .

- (b) Here are the application of BS and minimization to the regular expression  $R$  and the resulting deterministic minimal automaton  $A$  equivalent to  $R$ :

| $initials$    | $a_1$       | $c_2$    |
|---------------|-------------|----------|
| $\Sigma_{\#}$ | $followers$ |          |
| $a_1$         | $b_4$       | $\vdash$ |
| $c_2$         | $a_3$       |          |
| $a_3$         | $b_4$       | $\vdash$ |
| $b_4$         | $a_5$       |          |
| $a_5$         | $b_4$       | $\vdash$ |

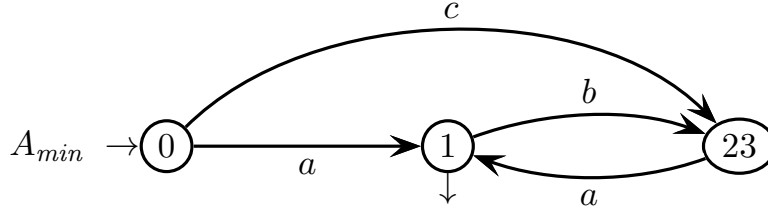


After state renaming, automaton  $A$  becomes:



Automaton  $A$  (with states renamed) is clean, as all the states are reachable and defined, and is deterministic by construction, yet it is not minimal. Clearly states 2 and 3 are undistinguishable, as both are non-final and have only an outgoing  $a$ -arc directed to the same (final) state. Therefore they can be merged into one state, say named 23. State 1 is the only final, thus it is distinguishable from all the others, and state 0 is the only one with an outgoing  $c$ -arc, thus it is also distinguishable from all the others. No other merge of states is possible.

Here is the resulting minimal automaton  $A_{min}$  equivalent to  $A$ :



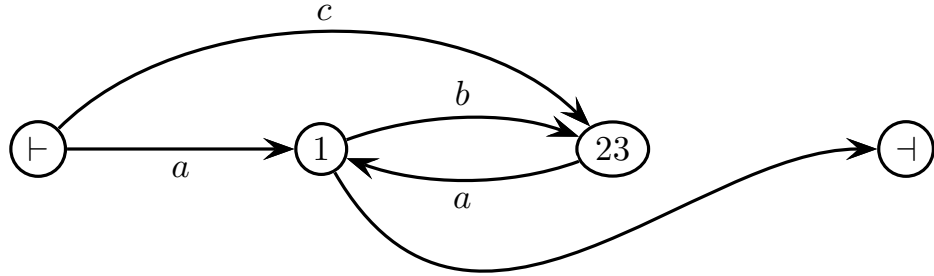
- (c) Here are the computations of automaton  $A_{min}$ , resulting from point b), for the two valid strings  $c a b a$  and  $a b a b a$ :

$$\begin{aligned} & \xrightarrow{\text{start}} q_0 \xrightarrow{c} q_{23} \xrightarrow{a} q_1 \xrightarrow{b} q_{23} \xrightarrow{a} q_1 \xrightarrow{\text{exit}} \\ & \xrightarrow{\text{start}} q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_{23} \xrightarrow{a} q_1 \xrightarrow{b} q_{23} \xrightarrow{a} q_1 \xrightarrow{\text{exit}} \end{aligned}$$

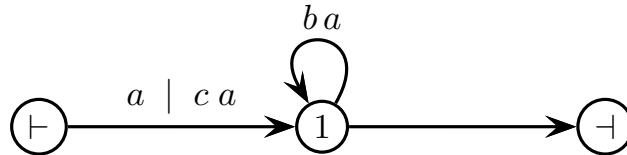
- (d) Here is the right-linear grammar (axiom  $Q_0$ ) derived from automaton  $A_{min}$  (and equivalent to both automata  $A_{min}$  and  $A$ ):

$$\left\{ \begin{array}{l} Q_0 \rightarrow a Q_1 \mid c Q_{23} \\ Q_1 \rightarrow b Q_{23} \mid \varepsilon \\ Q_{23} \rightarrow a Q_1 \end{array} \right.$$

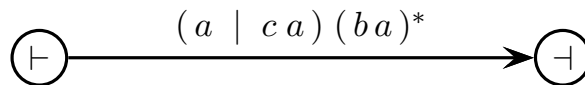
- (e) Here are the application of BMC (node elimination) to automaton  $A_{min}$  and the obtained equivalent RE. First normalize the end nodes (because the final node 1 has an outgoing arc – while the initial node is only renamed):



Then eliminate the inner node 23:



Finally eliminate the inner node 1:



The obtained RE turns out to be identical to  $R$  (a different elimination order may yield a different RE, though equivalent).

(f) Yes, the regular language  $L(R)$  is local. This can be justified in two ways:

- The Berry-Sethi automaton  $A$  that accepts such a language  $L(R)$  is local, as all the transitions labeled by the same input letter enter the same state. This feature is visible for both automaton  $A$ , where each state (but the initial one) is entered by exactly one letter, and for automaton  $A_{min}$ , where state 23 is entered by two letters, i.e.,  $a$  and  $c$ .
- The local sets of language  $L(R)$ , i.e.,  $Ini = \{ a, c \}$ ,  $Fin = \{ a \}$  and  $Dig = \{ ca, ab, ba \}$ , are such that all the strings necessarily start with either a letter  $a$  or a letter digram  $ca$ , possibly followed by a letter digram  $ab$  repeated two or more times. But such strings are precisely those generated by the regular expression  $R = (a \mid ca)(ba)^*$ , therefore language  $L(R)$  is local.

## 2 Free Grammars and Pushdown Automata 20%

1. Consider a language  $L$  defined over the four-letter alphabet  $\Sigma = \{ a, b, c, d \}$  and composed of nested pairs:

$$\underbrace{a^m \overbrace{c^n d^n} b^m}$$

with exponents  $m \geq 1$  and  $n \geq 0$ .

Here are two sample *valid* strings:

$$a a c c c d d d b b \quad a a b b$$

and two sample *invalid* strings:

$$c c d d \quad a a c d$$

Now consider a language  $L_1$  composed by all the words  $w \in \Sigma^*$  such that  $w b \in L$ , and then consider another language  $L_2 = L \cup L_1$ .

Answer the following questions:

- (a) List all the strings  $z$  of language  $L_2$  such that  $|z| \leq 3$ .
- (b) Write a non-ambiguous *BNF* grammar  $G$  that generates language  $L_2$ .
- (c) Draw the syntax trees of  $G$  for these two valid strings of language  $L_2$ :

$$a a c d b \quad a c d b$$

---

## Solution

- (a) Here are the strings of length  $\leq 3$  of language  $L_2$ :

$a \quad ab \quad acd \quad aab$

- (b) A possible non-ambiguous *BNF* grammar  $G$  for language  $L_1$  (axiom  $S$ ):

$$G \left\{ \begin{array}{l} S \rightarrow a S b \mid a C b \mid a C \\ C \rightarrow c C d \mid \varepsilon \end{array} \right.$$

In a systematic way, first write the grammars of  $L$  and  $L_1$ , separately:

$$G_L \left\{ \begin{array}{l} S_L \rightarrow a S_L b \mid a C_L b \\ C_L \rightarrow c C_L d \mid \varepsilon \end{array} \right.$$

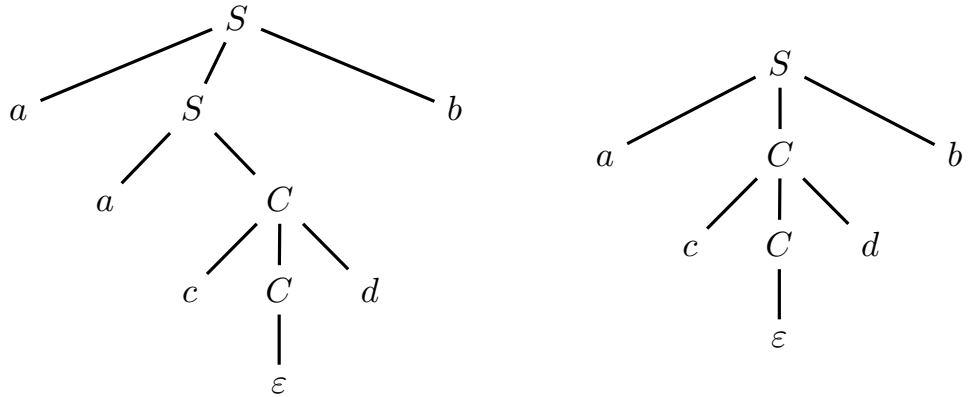
$$G_{L_1} \left\{ \begin{array}{l} S_{L_1} \rightarrow a S_{L_1} b \mid a C_{L_1} \\ C_{L_1} \rightarrow c C_{L_1} d \mid \varepsilon \end{array} \right.$$

and then write their union grammar, for  $L_2 = L \cup L_1$ :

$$G_{L \cup L_1} \left\{ \begin{array}{l} \frac{S \rightarrow S_L \mid S_{L_1}}{S_L \rightarrow a S_L b \mid a C_L b} \\ \frac{C_L \rightarrow c C_L d \mid \varepsilon}{S_{L_1} \rightarrow a S_{L_1} b \mid a C_{L_1}} \\ C_{L_1} \rightarrow c C_{L_1} d \mid \varepsilon \end{array} \right.$$

Grammar  $G_{L \cup L_1}$  contains a few structurally identical rules. After some simplification (here omitted), it becomes identical to grammar  $G$ .

- (c) Syntax trees of  $G$  for the two valid strings  $aacdb$  and  $acdb$ :



For completeness, here are the corresponding derivations:

$$S \Rightarrow a S b \Rightarrow a a C b \Rightarrow a a c C d b \Rightarrow a a c d b$$

$$S \Rightarrow a C b \Rightarrow a c C d b \Rightarrow a c d b$$

2. Consider a language  $L$  of nested lists, of arbitrary depth, specified as follows:

- there are lists of two types: A and B
- a list of type A has initial symbol **begin**, final symbol **end**, element *separator* comma “,”, and may be empty
- a list of type B has initial symbol **start**, final symbol **stop**, element *terminator* colon “:”, and may not be empty
- an element of a list of type A can be an identifier **id**, or a sublist of type A or B
- an element of a list of type B can be a number **num**, or a sublist of type B only
- identifiers **id** and numbers **num** are terminals and do not need to be further expanded
- a language phrase always has a single list (of type A or B) at the top level

Here is one sample valid phrase of language  $L$  (continues on the next line):

```
begin id, begin id end, id, id, start num : num : stop,  
begin end, start num : start num : stop : num : stop end
```

Answer the following questions:

- (a) Write an EBNF grammar  $G$ , non-ambiguous, that generates the language  $L$  specified above.
  - (b) Suppose to refine the specification of language  $L$  and cancel the separator (comma or colon, whichever it is) that occurs in front of every initial symbol **start**; call  $L'$  the so-refined language. Write an EBNF grammar  $G'$ , non-ambiguous, that generates language  $L'$  (if you wish, you can think of modifying grammar  $G$ ).
  - (c) (optional) In the original language  $L$ , cancel both the element terminator colon “:” and the final symbol **stop** of the lists of type B; call  $L''$  this other language. Write an EBNF grammar  $G''$ , non-ambiguous, that generates language  $L''$ . Is such a grammar still structurally adequate for nested lists? Justify your answer.
-



## Solution

- (a) Here is an EBNF grammar  $G$  (axiom LANG), which uses well-known list models:

$$\left\{ \begin{array}{l} \langle \text{LANG} \rangle \rightarrow \langle \text{LST\_A} \rangle \mid \langle \text{LST\_B} \rangle \\ \langle \text{LST\_A} \rangle \rightarrow \text{begin} \left[ \langle \text{ELM\_A} \rangle (', ' \langle \text{ELM\_A} \rangle)^* \right] \text{end} \\ \langle \text{LST\_B} \rangle \rightarrow \text{start} ( \langle \text{ELM\_B} \rangle ' : ' )^+ \text{stop} \\ \langle \text{ELM\_A} \rangle \rightarrow \langle \text{LST\_A} \rangle \mid \langle \text{LST\_B} \rangle \mid \text{id} \\ \langle \text{ELM\_B} \rangle \rightarrow \langle \text{LST\_B} \rangle \mid \text{num} \end{array} \right.$$

where the square brackets indicate optionality. All the used rule models for lists are known to be non-ambiguous. There is neither double-sided recursion, nor concatenation (or star) ambiguity, nor union ambiguity, nor circular derivation. Thus grammar  $G$  is reasonably non-ambiguous. Alternatively, grammar  $G$  is easily seen to be ELL(1), thus surely it is non-ambiguous.

- (b) Here is a possible grammar  $G'$  (derived from  $G$ ):

$$\left\{ \begin{array}{l} \langle \text{LANG} \rangle \rightarrow \langle \text{LST\_A} \rangle \mid \langle \text{LST\_B} \rangle \\ \langle \text{LST\_A} \rangle \rightarrow \text{begin} \\ \quad \left[ \langle \text{ELM\_A} \rangle (', ' ( \langle \text{LST\_A} \rangle \mid \text{id} ) \mid \langle \text{LST\_B} \rangle )^* \right] \\ \quad \text{end} \\ \langle \text{LST\_B} \rangle \rightarrow \text{start} \langle \text{ELM\_B} \rangle ( \langle \text{LST\_B} \rangle \mid ' : ' \text{num} )^* ' : ' \text{stop} \\ \langle \text{ELM\_A} \rangle \rightarrow \langle \text{LST\_A} \rangle \mid \langle \text{LST\_B} \rangle \mid \text{id} \\ \langle \text{ELM\_B} \rangle \rightarrow \langle \text{LST\_B} \rangle \mid \text{num} \end{array} \right.$$

Only two original rules of  $G$ , for LST\_A and LST\_B, have been changed. Grammar  $G'$  is not ambiguous either.

- (c) To obtain  $G''$ , it is tempting to jump into changing the rule of LST\_B as follows:

$$\langle \text{LST\_B} \rangle \rightarrow \text{start} \langle \text{ELM\_B} \rangle^+$$

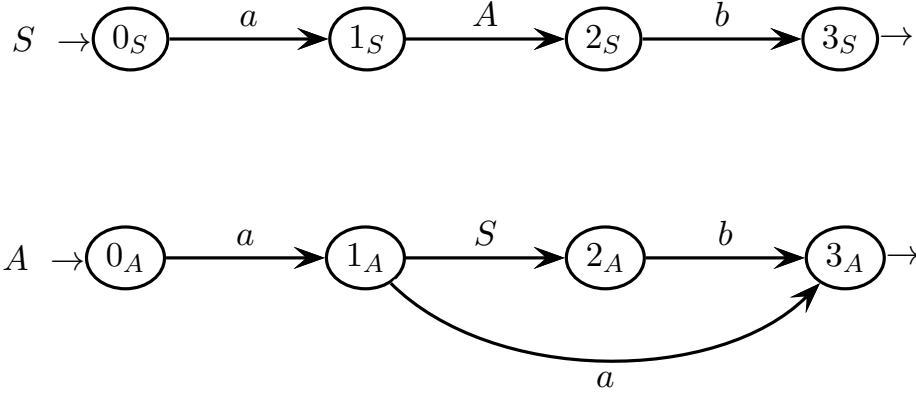
by just canceling colon “:” and stop. But that’s going too far. Since there is an alternative  $\langle \text{ELM\_B} \rangle \rightarrow \langle \text{LST\_B} \rangle$ , we immediately have the (expanded) rule  $\langle \text{LST\_B} \rangle \rightarrow \text{start} \langle \text{LST\_B} \rangle^+$ , which is (heavily) ambiguous as it uses two different mechanisms (iteration and right recursion) for the same purpose of generating a list. Thus we need to drop one mechanism, at our choice, for instance recursion (as type-B lists are now a regular language), and only use iteration as follows:

$$\langle \text{LST\_B} \rangle \rightarrow \text{start} ( \text{start} \mid \text{num} )^+ \text{num}$$

This rule is not ambiguous. Replace it into grammar  $G$  and so obtain a correct grammar  $G''$ , not ambiguous. Of course, now the (sub)tree of a type-B list is flattened and does not model the sublists as different tree branches, thus grammar  $G''$  is structurally inadequate. If we drop iteration instead, we obtain a linearized tree, still with no branches. For such poorly delimited lists, which just have an initial symbol, only a semantic check can tell which ends where. In conclusion, though ambiguity is avoidable for  $G''$ , structural adequacy is impossible.

### 3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar  $G$ , presented as a machine net, over the two-letter terminal alphabet  $\{ a, b \}$  and the nonterminal alphabet  $\{ A, S \}$  (axiom  $S$ ):



Answer the following questions (use the figures / tables / spaces on the next pages):

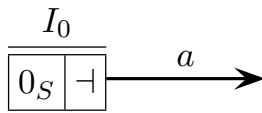
- (a) Draw the syntax trees of grammar  $G$  for these two valid sentences:

$a a a b \quad a a a a b b b$

- (b) Draw the pilot automaton and argue that grammar  $G$  is ELR (1).
  - (c) Write a simulation of the bottom-up (ELR) analysis of the strings of point (a).
  - (d) Determine if grammar  $G$  is of type ELL ( $k$ ), for some  $k \geq 1$ , by finding the guide sets on the call arcs and exit arrows, and by examining the possible conflicts.
  - (e) (optional) Write the recursive procedure for the top-down (i.e., recursive-descent) analysis for nonterminal  $A$ .
-

please draw here the syntax trees of strings  $a a a b$  and  $a a a a b b b$  – question (a)

please draw here the pilot of grammar  $G$  – question (b)



write here your comments to the pilot

please write here the two simulations of the bottom-up analysis – question (c)

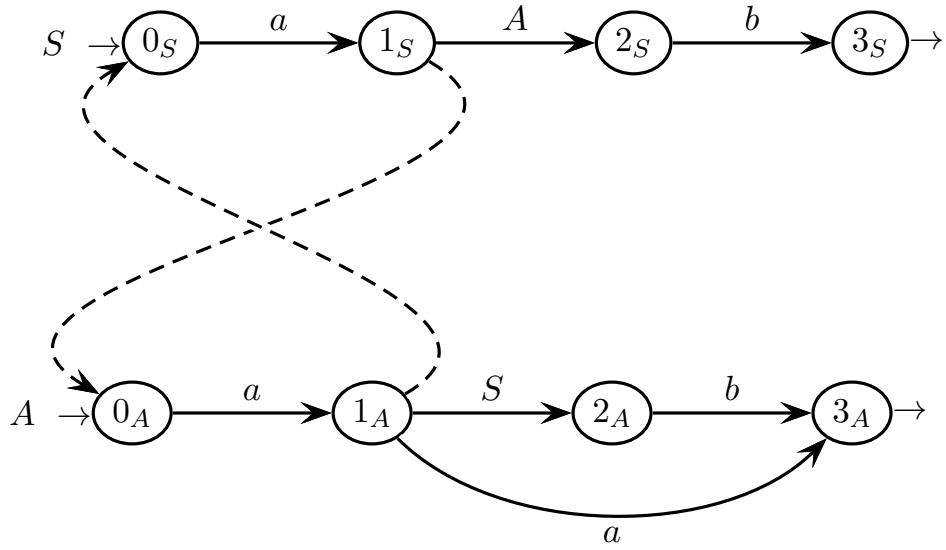
first ELR simulation table – string  $a a a b$  – (number of rows not significant)

[illegible]

second ELR simulation table – string  $a a a a b b b$  – (number of rows not significant)

[illegible]

please write here the guide sets – question (d)



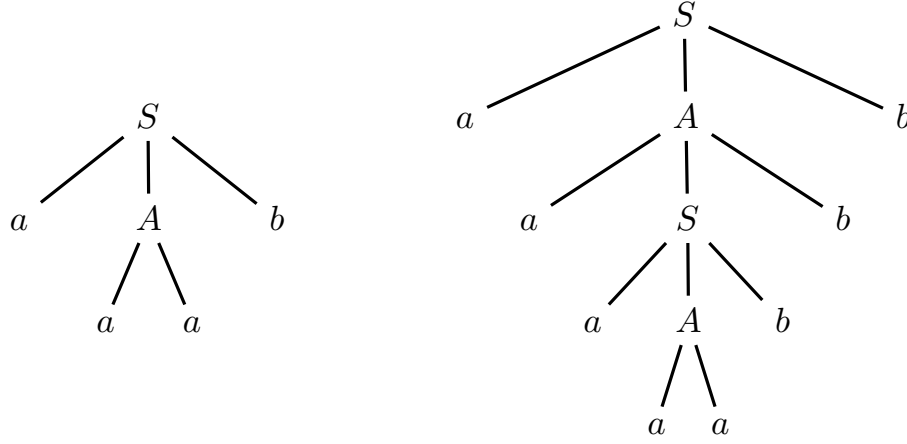
write here your comments to the guide sets

top-down (recursive-descent) analyzer for nonterminal  $A$  – question (e)

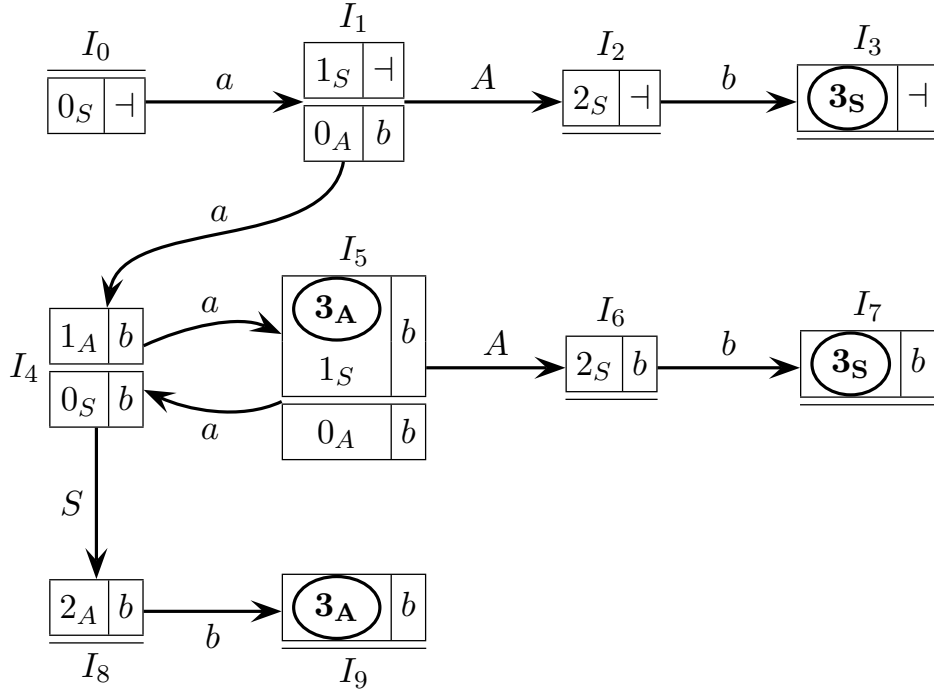


## Solution

(a) Syntax trees of the two strings  $aaab = a^3b$  and  $aaaaabbb = a^5b^3$ :



(b) Here is the pilot automaton of grammar  $G$ , with 10 m-states:



In this pilot, RR conflicts are impossible, as no m-state contains two (or more) final items. Each m-state  $I_3$ ,  $I_7$  and  $I_9$  contains a final item, but there are not any SR conflicts herein, as such m-states do not have any terminal shift outgoing transition. Also the m-state  $I_5$  contains a final item, but with a look-ahead  $b$ , and such a m-state does not have a  $b$ -shift outgoing transition, thus there is not a SR conflict here either. There is a double transition, i.e.,  $I_4 \xrightarrow{a} I_5$ , but the two destination net states in the m-state  $I_5$ , i.e.,  $3_A$  and  $1_S$ , are different, thus such a transition is not convergent. Since all the other pilot transitions are single, there are not any Convergence conflicts either. In conclusion, grammar  $G$  is ELR (1).

- (c) Bottom-up (ELR) simulation for the two sample strings (a few steps of reduction and the consecutive nonterminal shift are condensed):

first ELR simulation table – string  $a a a b$

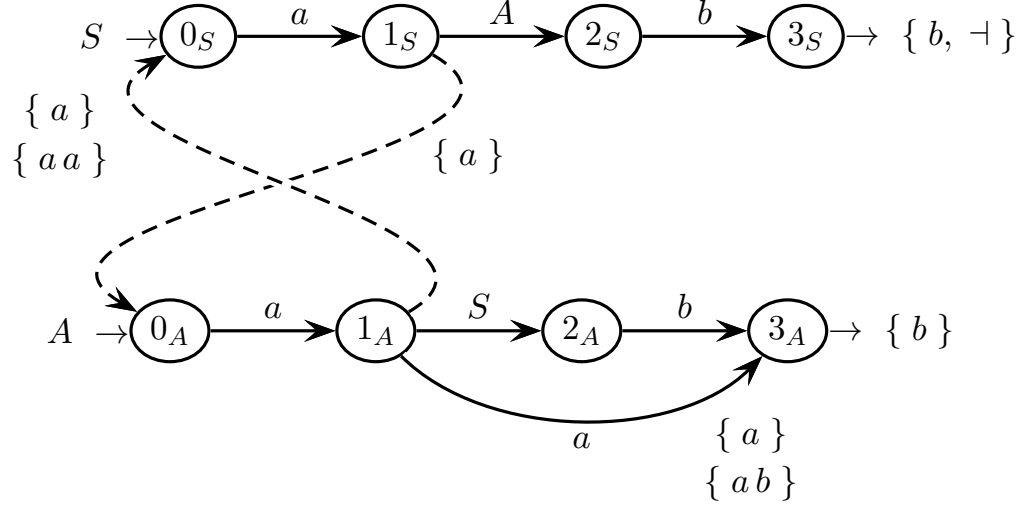
| <i>stack of m-states<br/>and grammar symbols</i> | <i>input left<br/>to be scanned</i> | <i>move executed</i>                                                  |
|--------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------|
| $I_0$                                            | $a a a b \dashv$                    | initial configuration                                                 |
| $I_0 a I_1$                                      | $a a b \dashv$                      | terminal shift: $I_0 \xrightarrow{a} I_1$                             |
| $I_0 a I_1 a I_4$                                | $a b \dashv$                        | terminal shift: $I_1 \xrightarrow{a} I_4$                             |
| $I_0 a I_1 a I_4 a I_5$                          | $b \dashv$                          | terminal shift: $I_4 \xrightarrow{a} I_5$                             |
| $I_0 a I_1 A I_2$                                | $b \dashv$                          | red.: $a a \rightsquigarrow A$ ; n.t.shift: $I_1 \xrightarrow{A} I_2$ |
| $I_0 a I_1 A I_2 b I_3$                          | $\dashv$                            | terminal shift: $I_2 \xrightarrow{b} I_3$                             |
| $I_0 S$                                          | $\dashv$                            | reduce: $a A b \rightsquigarrow S$ ; accept                           |

second ELR simulation table – string  $a a a a b b b$

| <i>stack of m-states<br/>and grammar symbols</i> | <i>input left<br/>to be scanned</i> | <i>move executed</i>                                                    |
|--------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------|
| $I_0$                                            | $a a a a a b b b \dashv$            | initial configuration                                                   |
| $I_0 a I_1 a I_4 a I_5 a I_4 a I_5$              | $b b b \dashv$                      | terminal shifts: $\left(I_0 \xrightarrow{a a a a a} I_5\right)^a$       |
| $I_0 a I_1 a I_4 a I_5 A I_6$                    | $b b b \dashv$                      | red.: $aa \rightsquigarrow A$ ; n.t.shift: $I_5 \xrightarrow{A} I_6$    |
| $I_0 a I_1 a I_4 a I_5 A I_6 b I_7$              | $b b \dashv$                        | terminal shift: $I_6 \xrightarrow{b} I_7$                               |
| $I_0 a I_1 a I_4 S I_8$                          | $b b \dashv$                        | red.: $a A b \rightsquigarrow S$ ; n.t.shift: $I_4 \xrightarrow{S} I_8$ |
| $I_0 a I_1 a I_4 S I_8 b I_9$                    | $b \dashv$                          | terminal shift: $I_8 \xrightarrow{b} I_9$                               |
| $I_0 a I_1 A I_2$                                | $b \dashv$                          | red.: $a S b \rightsquigarrow A$ ; n.t.shift: $I_1 \xrightarrow{A} I_2$ |
| $I_0 a I_1 A I_2 b I_3$                          | $\dashv$                            | terminal shift: $I_2 \xrightarrow{b} I_3$                               |
| $I_0 S$                                          | $\dashv$                            | reduce $a A b \rightsquigarrow S$ ; accept                              |

<sup>a</sup>This is a series of five terminal shifts, reductions and nonterminal shifts, which in total scan the input substring  $a a a a a = a^5$ .

(d) Here are the guide sets, for  $k = 1$  and  $k = 2$  (where it helps):



Concerning ELL (1). The grammar is not ELL (1) because of the conflict between the guide sets on the arcs exiting from state  $1_A$ .

Concerning ELL (2). The grammar is ELL (2) because the guide sets of length 2 on the arcs exiting from state  $1_A$ , i.e.,  $\{aa\}$  and  $\{ab\}$ , are disjoint.

- (e) Here is the top-down (recursive-descent) analyzer for nonterminal  $A$ . Since the grammar is ELL (2), the procedure uses two variables, i.e.,  $cc1$  and  $cc2$ , for the first and second symbol in the input tape, respectively: var  $cc1$  is the current symbol and var  $cc2$  is the look-ahead one. It is assumed that the auxiliary procedure  $next$  updates both: it performs the assignment  $cc1 := cc2$  and reads  $cc2$  from the input tape, i.e., it shifts the input sliding window.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>procedure</b> <math>A</math> <b>if</b> <math>cc1 = a</math> <b>then</b>                                // state <math>0_A</math>       <math>cc1, cc2 := next</math>                               // <math>0_A \xrightarrow{a} 1_A</math>     <b>if</b> <math>cc1, cc2 = a, a</math> <b>then</b>                       // state <math>1_A</math>               <b>call</b> <math>S</math>                                       // <math>1_A \xrightarrow{S} 2_A</math>         <b>if</b> <math>cc1 = b</math> <b>then</b>                             // state <math>2_A</math>                       <math>cc1, cc2 := next</math>                         // <math>2_A \xrightarrow{b} 3_A</math>             <b>if</b> <math>cc1 = b</math> <b>then</b>                         // state <math>3_A</math>                               <b>return</b>                               // <math>3_A \xrightarrow{\text{exit}}</math>                           <b>else error</b>                             // state <math>3_A</math>                   <b>else error</b>                                 // state <math>2_A</math>           <b>else if</b> <math>cc1, cc2 = a, b</math> <b>then</b>                 // state <math>1_A</math>               <math>cc1, cc2 := next</math>                               // <math>1_A \xrightarrow{a} 3_A</math>         <b>if</b> <math>cc1 = b</math> <b>then</b>                             // state <math>3_A</math>                       <b>return</b>                               // <math>3_A \xrightarrow{\text{exit}}</math>                   <b>else error</b>                                 // state <math>3_A</math>           <b>else error</b>                                 // state <math>1_A</math>   <b>else error</b>                                       // state <math>0_A</math> <b>end procedure</b> </pre> | <pre> <b>procedure</b> <math>A</math> <b>if</b> <math>cc1 = a</math> <b>then</b>     <math>cc1, cc2 := next</math>     <b>if</b> <math>cc1, cc2 = a, a</math> <b>then</b>         <b>call</b> <math>S</math>         <b>if</b> <math>cc1 = b</math> <b>then</b>                       <math>cc1, cc2 := next</math>             <b>if</b> <math>cc1 = b</math> <b>then</b>                               <b>return</b>                   <b>else if</b> <math>cc1, cc2 = a, b</math> <b>then</b>                       <math>cc1, cc2 := next</math>             <b>return</b>           <b>error</b> <b>end procedure</b> </pre> |
| non-optimized encoding                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | optimized encoding                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

As usual, some optimization is possible, by grouping all the error cases at the procedure end and by suppressing some redundant checks.

## 4 Language Translation and Semantic Analysis 20%

1. Consider the grammar  $G$  below (axiom  $E$ ), that generates the language  $L$  of the arithmetic expressions in Polish (prefix) notation:

$$G: E \rightarrow o E E \mid n$$

over the (source) alphabet  $\Sigma = \{ o, n \}$ , where symbol  $o$  represents the operators and symbol  $n$  represents the integer numbers.

We want to define a translation  $\tau$  where the arithmetic expressions from  $L$  are rephrased in the infix notation, by alternating in depth two parenthesis types, namely  $()$  and  $[\ ]$ , i.e., round brackets and square brackets.

Here are two sample translations:

$$\tau(+ - 1 2 3) = ([1 - 2] + 3)$$

$$\tau(\times + - 1 2 3 \div 4 5) = ([ (1 - 2) + 3 ] \times [4 \div 5])$$

The target alphabet of the translated strings is  $\Delta = \{ n, o, (, ), [, ] \}$ .

Answer the following questions:

- (a) Design a translation grammar (or scheme)  $G'_\tau$  that defines translation  $\tau$ . You can suitably modify grammar  $G$ , if necessary.
- (b) Draw the source / target syntax tree(s) (unified or separate) of the translation grammar (or scheme)  $G'_\tau$  for the string  $w$  below:

$$w = o n o n n$$

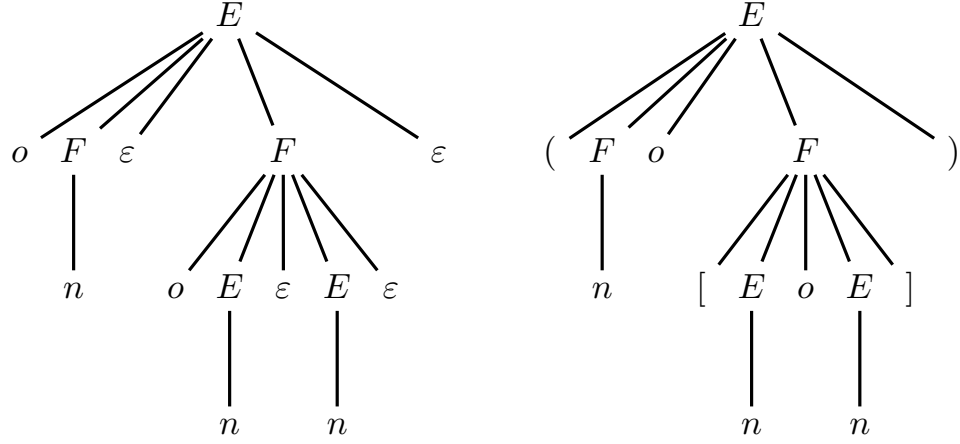
- (c) (optional) Say if translation  $\tau$  can be computed by a top-down parser, i.e., recursive-descent or ELL, enriched with write actions, and justify your answer. If translation  $\tau$  can be computed by a recursive-descent parser, write one.

## Solution

- (a) Here is the translation grammar  $G'_\tau$  for translation  $\tau$  (axiom  $E$ ), obtained by doubling the nonterminal  $E$  into  $E$  and  $F$ , in order to distinguish between subexpressions at even and odd depth level, respectively:

$$G'_\tau \left\{ \begin{array}{l} E \rightarrow \frac{o}{\left(} F \frac{\varepsilon}{o} F \frac{\varepsilon}{\right)} \mid \frac{n}{n} \\ F \rightarrow \frac{o}{\left[} E \frac{\varepsilon}{o} E \frac{\varepsilon}{\right]} \mid \frac{n}{n} \end{array} \right.$$

- (b) Here are the trees of  $G'_\tau$ , source (left) and destination (right) separately for the source string  $w = ononn$  and the corresponding translation  $\tau(w)$ :



For completeness, here is the (leftmost) derivation for the source string  $w = ononn$  along with the corresponding translation:

$$E \Rightarrow \frac{o}{\left(} F \frac{\varepsilon}{o} F \frac{\varepsilon}{\right)} \Rightarrow \frac{o}{\left(} \frac{n}{n} \frac{\varepsilon}{o} F \frac{\varepsilon}{\right)} \Rightarrow \frac{o}{\left(} \frac{n}{n} \frac{\varepsilon}{o} \frac{o}{\left[} E \frac{\varepsilon}{o} E \frac{\varepsilon}{\right]} \frac{\varepsilon}{\right)} \xRightarrow{2} \frac{o}{\left(} \frac{n}{n} \frac{\varepsilon}{o} \frac{o}{\left[} \frac{n}{n} \frac{\varepsilon}{o} \frac{n}{\right]} \frac{\varepsilon}{\right)} \frac{\varepsilon}{\right)}$$

and here are the projections over the source and target alphabets  $\Sigma$  and  $\Delta$ :

$$h_\Sigma = ononn \quad h_\Delta = (no[non])$$

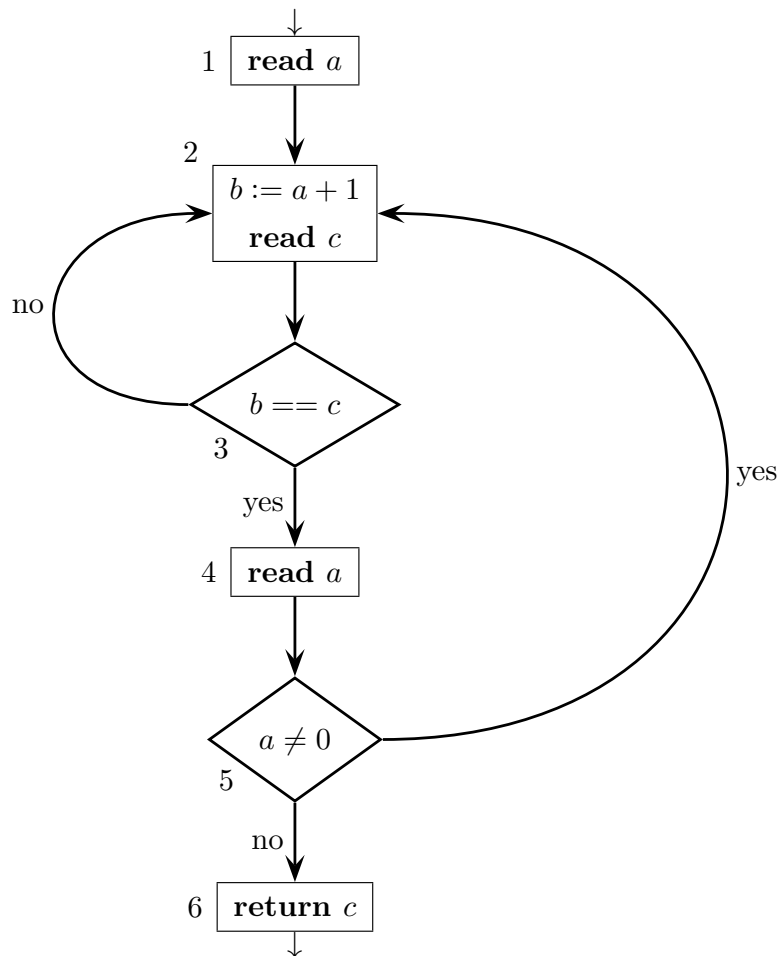
Of course, a rightmost derivation would yield the same result.

- (c) The source grammar  $G'_s$  does not exhibit left recursion and all the alternative source rules for each non-terminal begin with a different terminal, e.g.,  $E \rightarrow o F F$  and  $E \rightarrow n$ , and similarly for  $F$ . As such, a simple implementation of the translation grammar can be obtained as follows:

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>procedure</b> <math>E</math> <b>if</b> <math>cc = o</math> <b>then</b>   <b>write</b> ( '(' )   <math>cc := next</math>   <b>if</b> <math>cc \in \{ o, n \}</math> <b>then</b>     <b>call</b> <math>F</math>     <b>write</b> ( <math>o</math> )     <b>if</b> <math>cc \in \{ o, n \}</math> <b>then</b>       <b>call</b> <math>F</math>       <b>write</b> ( ' ) ' )       <b>if</b> <math>cc \in \{ o, n, \vdash \}</math> <b>then return</b>       <b>else error</b>     <b>else error</b>   <b>else if</b> <math>cc = n</math> <b>then</b>     <b>write</b> ( <math>n</math> )     <math>cc := next</math>     <b>if</b> <math>cc \in \{ o, n, \vdash \}</math> <b>then return</b>     <b>else error</b>   <b>else error</b> <b>end procedure</b> </pre> | <pre> 1 <b>def</b> <math>E</math> ( ): 2   <b>if</b> <b>accept</b> ( 'n' ): 3     <b>write</b> ( 'n' ) 4   <b>elif</b> <b>accept</b> ( 'o' ): 5     <b>write</b> ( '(' ) 6     <math>F</math> ( ) 7     <b>write</b> ( 'o' ) 8     <math>F</math> ( ) 9     <b>write</b> ( ' ) ' ) 10    <b>else: error</b> ( ) 11 12 <b>def</b> <math>F</math> ( ): 13   <b>if</b> <b>accept</b> ( 'n' ): 14     <b>write</b> ( 'n' ) 15   <b>elif</b> <b>accept</b> ( 'o' ): 16     <b>write</b> ( '[' ) 17     <math>E</math> ( ) 18     <b>write</b> ( 'o' ) 19     <math>E</math> ( ) 20     <b>write</b> ( ' ] ' ) 21   <b>else: error</b> ( ) </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

On the left, there is the pseudocode of the procedure for nonterminal  $E$ , with all guide set checks; the procedure for nonterminal  $F$  is similar. On the right, there is a Python encoding for both nonterminals, with some optimization.

2. Consider the program Control Flow Graph (CFG) shown below, with six nodes:



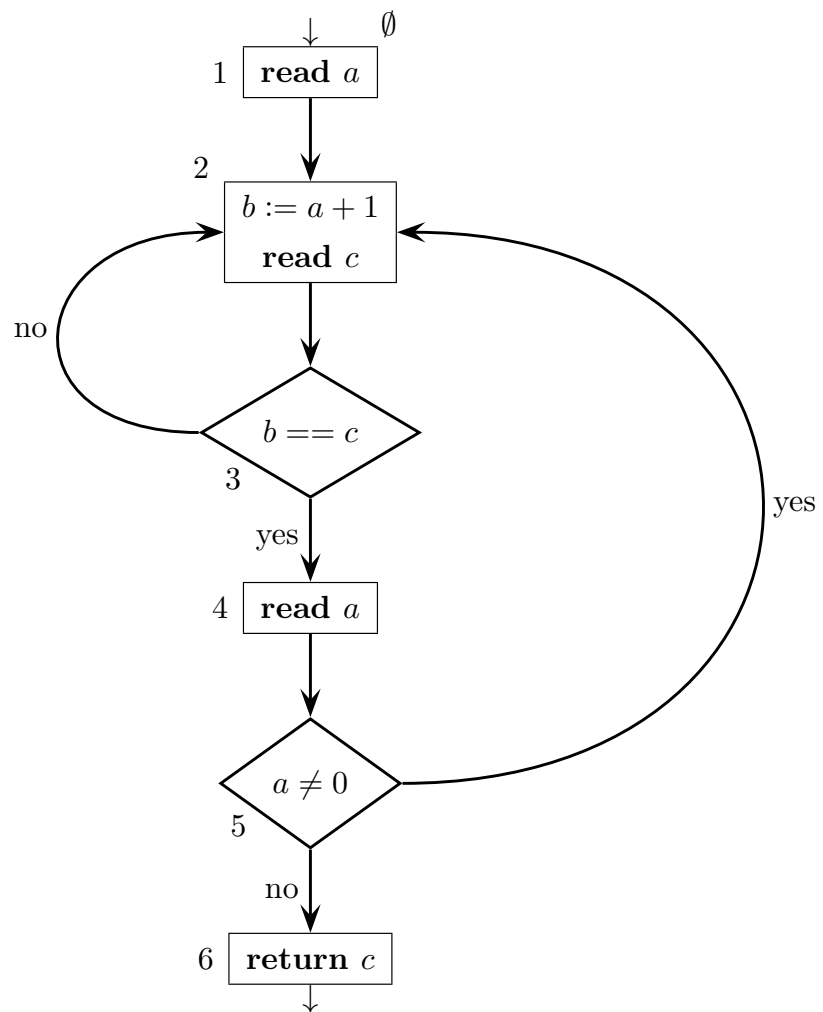
The program has three integer variables:  $a$ ,  $b$  and  $c$ . There are no input arguments. Answer the following questions (use the tables / drawings / spaces on the next pages):

- Disregarding semantics, write a (non-ambiguous) regular expression  $R$  that represents the execution traces of the CFG over the node alphabet  $\{1, \dots, 6\}$ .
- Informally find the live variables at each node and list them by each node (the live variables are all those live at the node input(s)). Can the found liveness intervals be exploited to optimize the memory allocation for the program?
- Find the variables used and defined at each node. Then write the flow equations of the live variables, but only for the nodes 2 and 5 (do not solve such equations). Remember that each node has two equations: *in* and *out*. Verify that the solution found at point (b) satisfies the *in* flow equations of nodes 2 and 5, in this way: show that by filling in your informal solution, the equations are satisfied.
- (optional) List all the variable definitions. Then informally find the reaching definitions at each node and list them by each node (the reaching definitions are all those live at the node output(s)). List also the suppressed variable definitions (specify both the suppressed and the “suppressing” definitions).



please write here the RE  $R$  – question (a)

please write here the live variables – question (b)



please comment here about the memory allocation

space for answering question (c)

tables of definitions and usages at the nodes

| <i>node</i> | <i>defined</i> | <i>node</i> | <i>used</i> |
|-------------|----------------|-------------|-------------|
| 1           |                | 1           |             |
| 2           |                | 2           |             |
| 3           |                | 3           |             |
| 4           |                | 4           |             |
| 5           |                | 5           |             |
| 6           |                | 6           |             |

(partial) system of data-flow equations

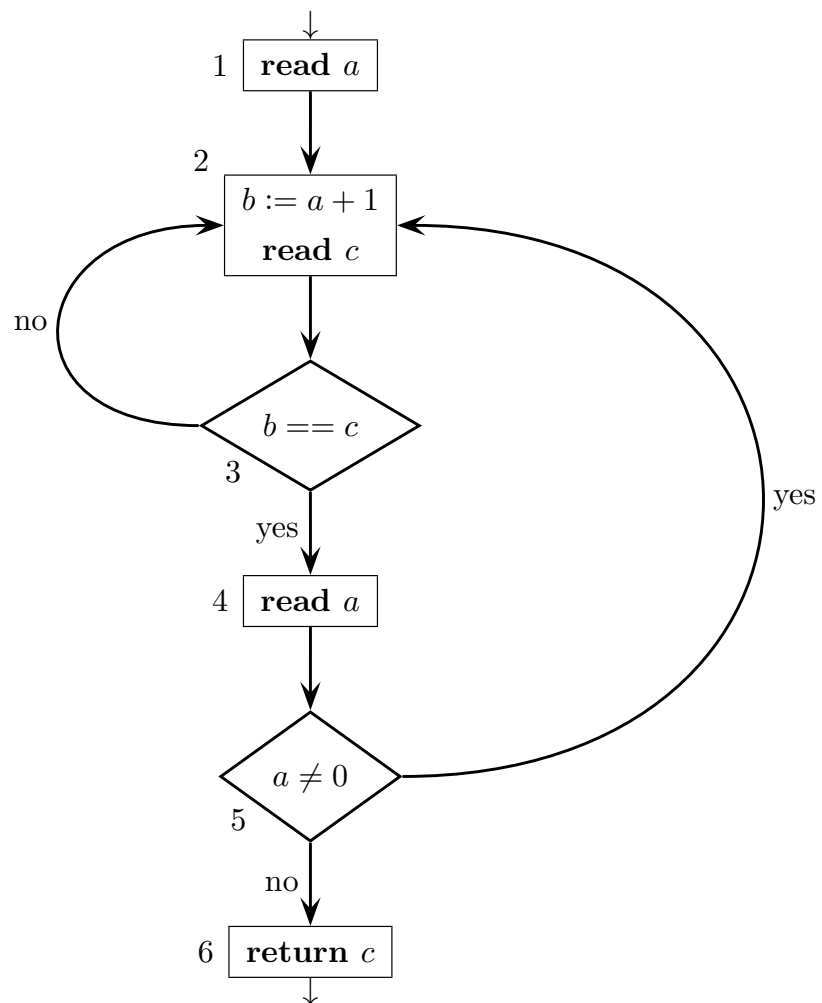
| <i>node</i> | <i>in equation</i> | <i>out equation</i> |
|-------------|--------------------|---------------------|
| 2           |                    |                     |
| 5           |                    |                     |

space for the equation verification

space for answering question (d)

please list here the variable definitions

please write here the reaching definitions



please list here the suppressed and “suppressing” definitions

## Solution

- (a) Here is a possible regular expression  $R$ :

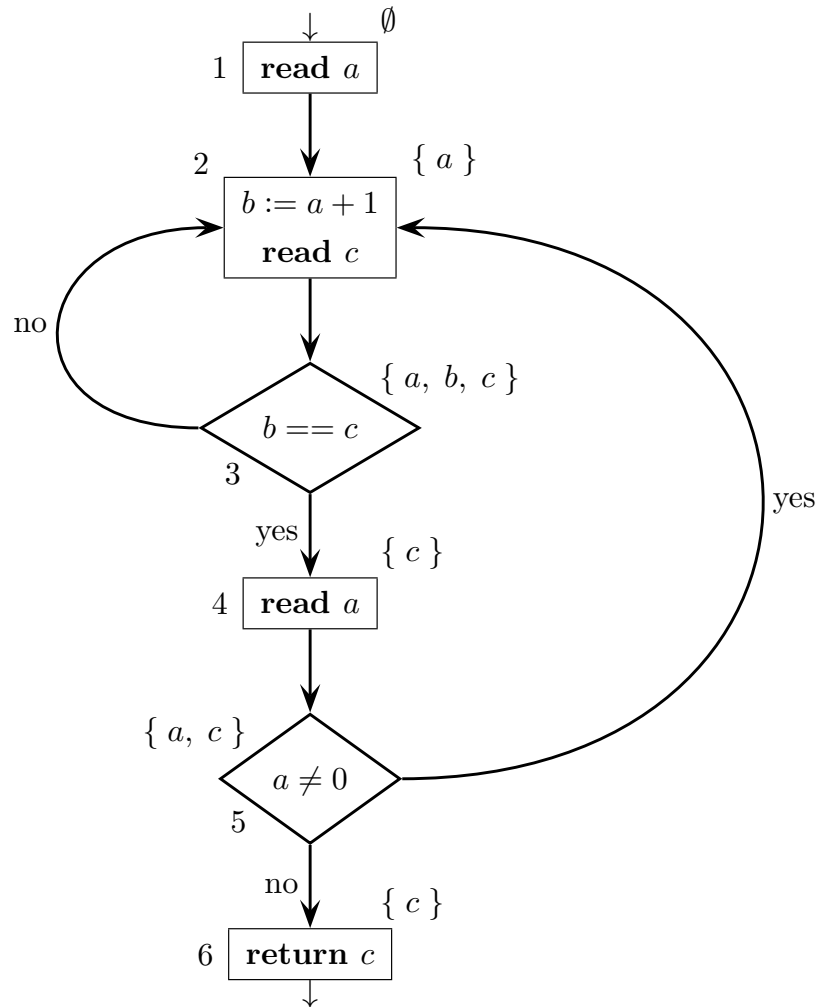
$$R = 1 \ (2 \ 3 \ [4 \ 5])^* \ 2 \ 3 \ 4 \ 5 \ 6$$

or also:

$$R = 1 \ ((2 \ 3)^+ \ 4 \ 5)^+ \ 6$$

The square brackets indicate optionality.

- (b) Here are the live variables (at the node input):



No pair of live variables happens to be mutually exclusive, therefore no memory optimization is possible.

(c) Flow equations and verification:

tables of definitions and usages at the nodes

| <i>node</i> | <i>defined</i> | <i>node</i> | <i>used</i> |
|-------------|----------------|-------------|-------------|
| 1           | $a$            | 1           | —           |
| 2           | $b\ c$         | 2           | $a$         |
| 3           | —              | 3           | $b\ c$      |
| 4           | $a$            | 4           | —           |
| 5           | —              | 5           | $a$         |
| 6           | —              | 6           | $c$         |

(partial) system of data-flow equations

| <i>node</i> | <i>in equation</i>                             | <i>out equation</i>         |
|-------------|------------------------------------------------|-----------------------------|
| 2           | $in(2) = \{ a \} \cup ( out(2) - \{ b, c \} )$ | $out(2) = in(3)$            |
| 5           | $in(5) = \{ a \} \cup ( out(5) - \emptyset )$  | $out(5) = in(2) \cup in(6)$ |

From the informal solution, question (b), we have:  $in(2) = \{ a \}$ ,  $in(3) = \{ b, c \}$ ,  $in(5) = \{ a, c \}$  and  $in(6) = \{ c \}$ . By replacing into the *out* equations we obtain:  $out(2) = in(3) = \{ a, b, c \}$  and  $out(5) = in(2) \cup in(6) = \{ a \} \cup \{ c \} = \{ a, c \}$ . Therefore, by replacing into the *in* equations we conclude that:

$$in(2) = \{ a \} \cup ( out(2) - \{ b, c \} )$$

becomes:

$$\{ a \} = \{ a \} \cup ( \{ a, b, c \} - \{ b, c \} ) = \{ a \} \cup \{ a \} = \{ a \}$$

which is an identity, and that:

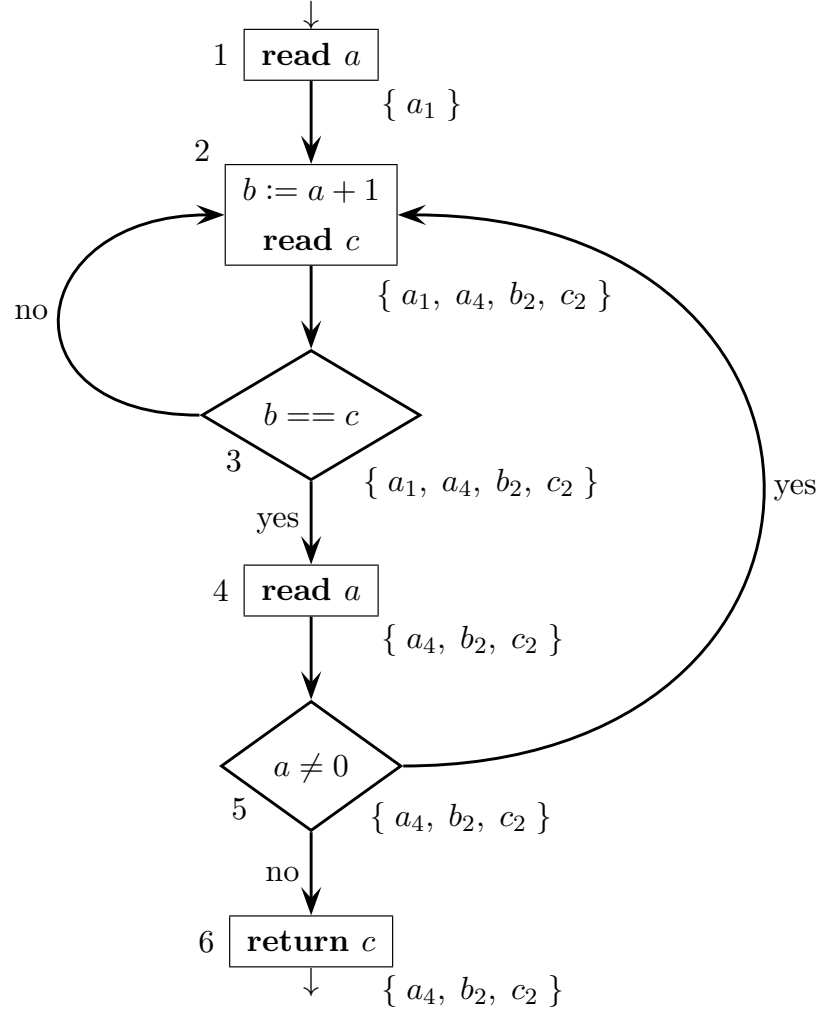
$$in(5) = \{ a \} \cup ( out(5) - \emptyset )$$

becomes:

$$\{ a, c \} = \{ a \} \cup ( \{ a, c \} - \emptyset ) = \{ a \} \cup \{ a, c \} = \{ a, c \}$$

which is also an identity. The solution (partial) verification is successful.

- (d) There are four variable definitions:  $a_1$ ,  $b_2$ ,  $c_2$  and  $a_4$ . Here are the reaching definitions (at the node output):



Clearly node 4 (def.  $a_4$ ) suppresses definition  $a_1$  and node 1 (def.  $a_1$ ) suppresses definition  $a_4$  (though definition  $a_4$  cannot reach node 1).